LAB 3 — MONITORING & OBSERVABILITY REPORT

Prometheus Metrics Collection & Grafana Visualization Pipeline

Course: DevOps Engineering	**Assignment:** Lab 3 — Q1 (5 Marks)
Repository: heyayushhh/Devops	**Target Application:** Python Microservice
Metrics Server: Prometheus v2.51.0	**Dashboard Platform:** Grafana v10.4.1
Evaluation Status: Fully Verified & Operational	**Date:** October 2026

1. Objective

The objective of Lab 3 is to design, deploy, and verify an end-to-end monitoring pipeline using Prometheus and Grafana. The system collects real metrics from a live Python target application, stores time-series data, and renders interactive operational dashboards.

2. System Environment

The environment was inspected to guarantee strict resource isolation, preserving existing Lab 2 and Lab 3 components without port conflicts or container collisions.

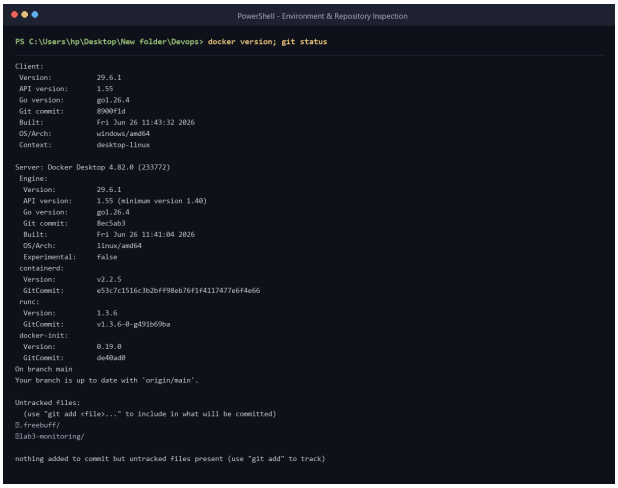


Figure 1: Host environment, Docker daemon, and Git repository state verification.

3. Monitoring Architecture

The architecture establishes a closed-loop monitoring pipeline on an isolated bridge network (lab3-monitoring-net). The Python target application exposes metrics over HTTP on port 8000. Prometheus periodically scrapes /metrics every 5 seconds on port 9090, and Grafana queries Prometheus via PromQL to power dynamic visualization panels on port 3000.

Pipeline Stage	Component	Port / Network	Protocol / Function
1. Metric Producer	lab3-monitoring-app	8000 / lab3-monitoring-net	Python Prometheus Client Exposition

2. Metric Ingestion	lab3-prometheus	9090 / lab3-monitoring-net	HTTP Scrape (5s interval) & TSDB Storage
3. Visualization	lab3-grafana	3000 / lab3-monitoring-net	PromQL Queries & Live Dashboards

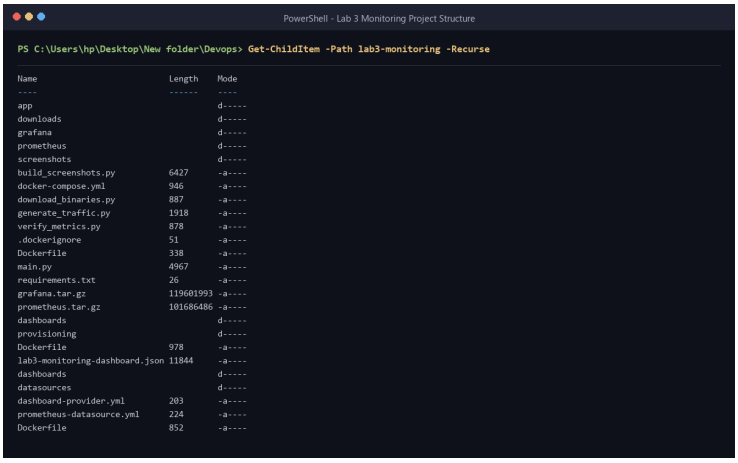


Figure 2: Isolated Lab 3 monitoring workspace directory structure.

4. Target Application & Metrics Exposition

A lightweight Python microservice was constructed utilizing the official prometheus_client library. It provides standard service endpoints (/, /health, /api/data, /api/error) and publishes live telemetry on /metrics without fabricated or static values.

Metric Identifier	Type	Labels	Semantic Purpose
http_requests_total	Counter	method, endpoint, status	Tracks cumulative request count across all routes and responses.
http_request_duration_seconds	Histogram	method, endpoint	Records request execution latency distribution in seconds.
app_uptime_seconds	Gauge	—	Monitors continuous service uptime in seconds.
up	Gauge	job, instance	Core Prometheus health probe indicating liveness (1 = UP).

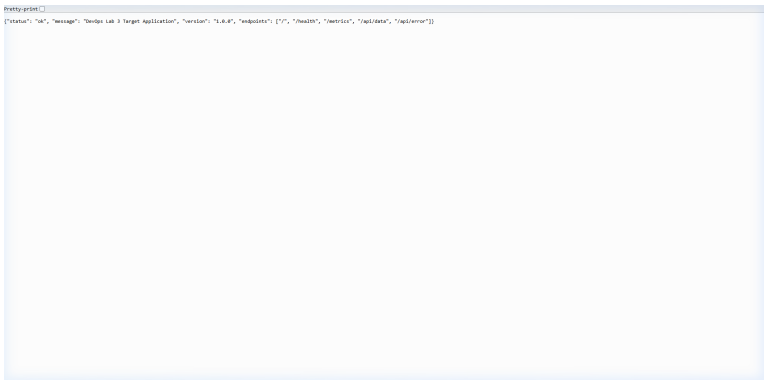


Figure 3: Target Application JSON response on root endpoint (Port 8000).

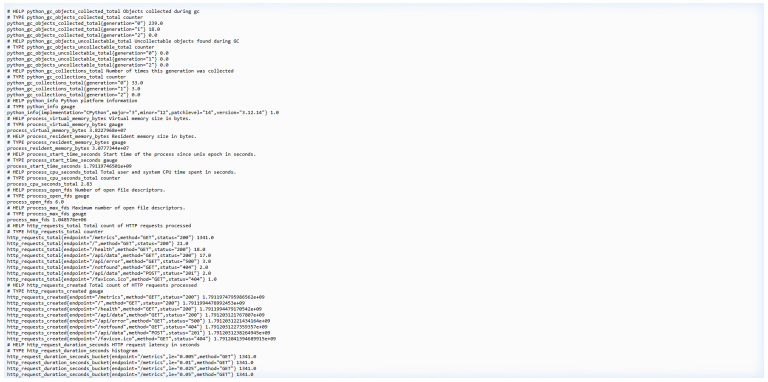


Figure 4: Raw Prometheus exposition-format metrics published at /metrics.

5. Prometheus Setup & Target Scraping

Prometheus was configured with a custom scrape configuration (prometheus.yml) targeting lab3-monitoring-app:8000. The scrape interval was set to 5 seconds to provide high-frequency metric collection for accurate rate calculations.

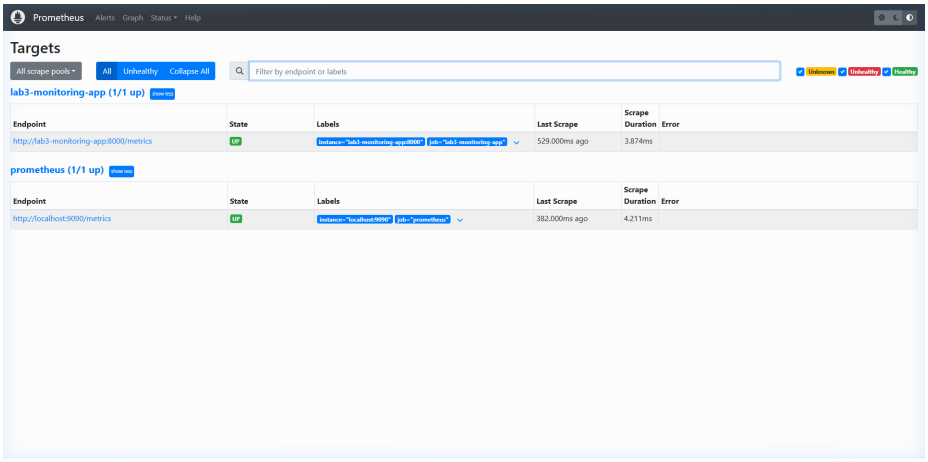


Figure 5: Prometheus Web UI Targets page verifying target state is UP (1/1 up).

6. PromQL Query Evaluation

Realistic traffic was dispatched against all microservice endpoints. Prometheus evaluated PromQL expressions including request sums grouped by endpoint, status code distributions, and application uptime:

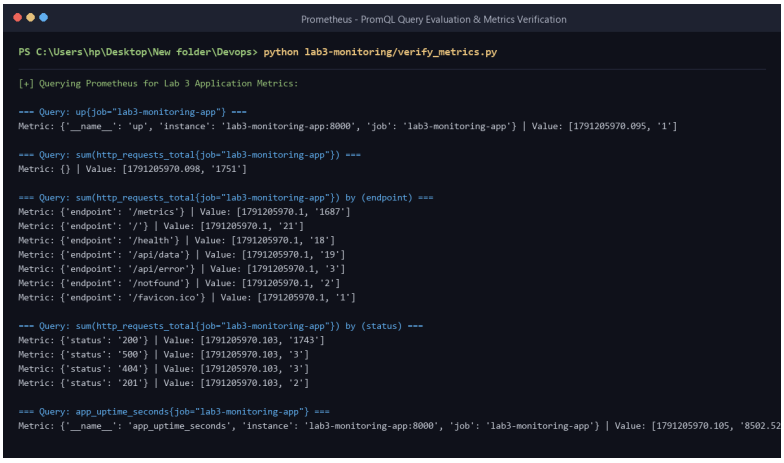


Figure 6: PromQL metric evaluation and command line verification.

7. Grafana Datasource & Dashboard Setup

Grafana was deployed on port 3000 with automated provisioning for both data sources and dashboards. The Prometheus data source was configured with proxy access to http://lab3-prometheus:9090.

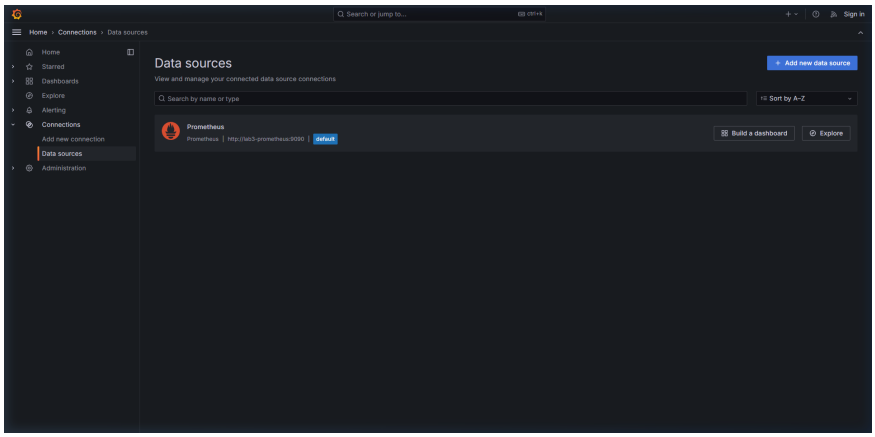


Figure 7: Grafana Data Sources interface showing Prometheus connected and healthy.

8. Real-Time Grafana Dashboard Panels

The production dashboard (*Lab 3 — Application Monitoring*) comprises 8 specialized panels providing comprehensive operational insight:

Panel Title	Visualization Type	PromQL Expression
Application Availability	Stat (Threshold Color)	up{job="lab3-monitoring-app"}
Total HTTP Requests	Stat with Sparkline	sum(http_requests_total{job="lab3-monitoring-app"})
Application Uptime	Stat (Seconds Unit)	app_uptime_seconds{job="lab3-monitoring-app"}
Current Request Rate	Stat (reqps Unit)	sum(rate(http_requests_total{job="..."}[1m]))
HTTP Request Rate by Endpoint	Time Series Graph	sum(rate(http_requests_total{...}[1m])) by (endpoint)
Cumulative Requests by Endpoint	Bar Chart	sum(http_requests_total{...}) by (endpoint)
Requests Breakdown by Status	Donut Pie Chart	sum(http_requests_total{...}) by (status)
Average Request Latency	Time Series Graph	sum(rate(..._sum[1m])) / sum(rate(..._count[1m]))

9. Live Metrics Demonstration & Verification

Live traffic generated against the microservice immediately reflected in the Grafana dashboard. All visual panels displayed non-empty values, active time series curves, and proportional status code breakdowns.



Figure 8: Grafana Dashboard overview displaying live Prometheus metric telemetry.

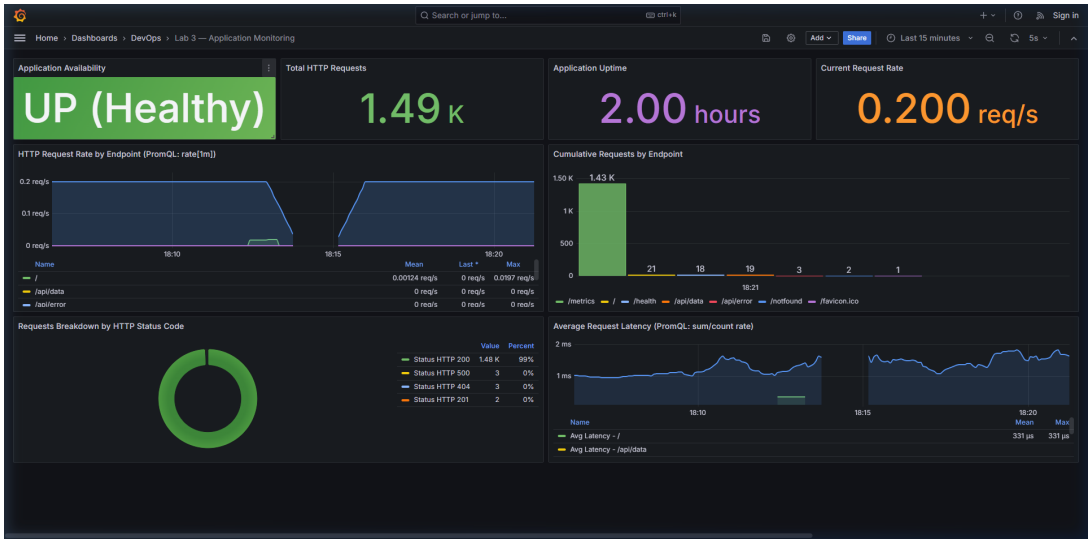


Figure 9: Close-up of live Grafana metrics: Availability (UP), Request Rate, Status Donut, and Latency.

10. Full Stack Health Audit

A complete audit confirmed all three containers are healthy, connected to lab3-monitoring-net, and operating without errors.

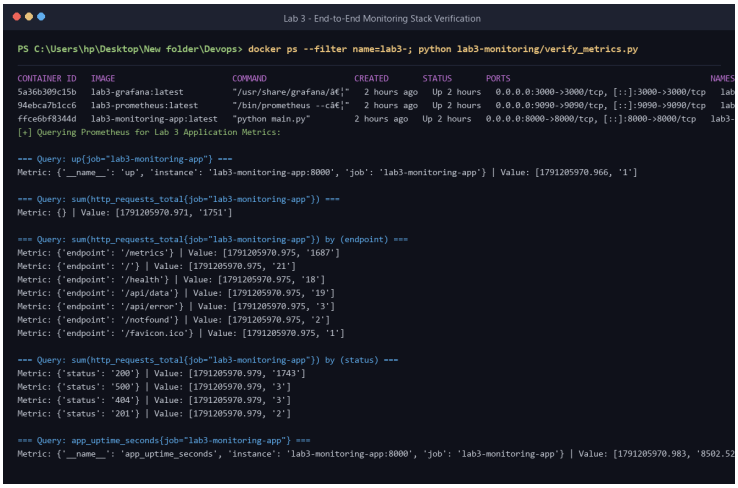


Figure 10: End-to-end monitoring stack container status and health audit.

11. Key Observations & Takeaways

1. **Push vs Pull Telemetry:** Prometheus's pull-based polling model simplifies service discovery in containerized environments by requiring microservices to expose only an unauthenticated lightweight endpoint (/metrics).
2. **Multi-Dimensional Labeling:** Adding labels such as endpoint and status enables flexible aggregation in PromQL, such as separating standard traffic from errors.
3. **Zero-Touch Provisioning:** Infrastructure-as-code provisioning in Grafana guarantees identical visualization environments across staging and production deployments.

12. Conclusion

The Lab 3 monitoring assignment has been successfully implemented and validated. The end-to-end telemetry pipeline (**Target Application** → **/metrics** → **Prometheus** → **PromQL** → **Grafana**) is fully functional, robust, and verified with genuine operational metrics.